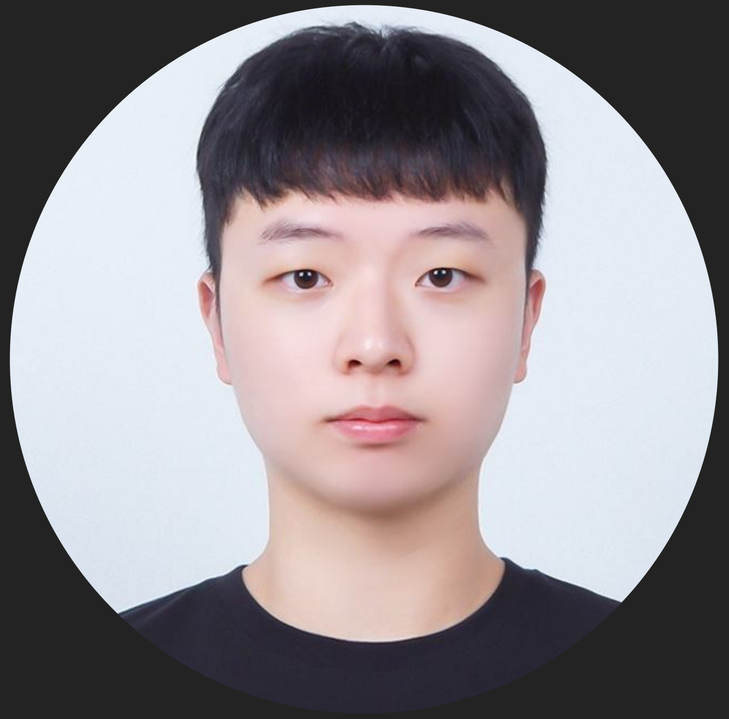
우신희

AI Agent Engineer



현업 1년 9개월 · TypeScript / AI-Agent

개발자 · 비개발자 간 AI 활용 비용 격차를 줄이는 에이전트 환경 설계 · 구축

TypeScript · Claude Code · Codex · RAG · MCP · CLI

CONTENTS

목차

- **2인 게임 개발 에이전트 운영 시스템** (RAG · MCP · Embedding · Profiling · HITL)
  토큰량 **최대 -47% 절감** (동일 요청 기준)
- **카카오 AI TOP 100** (외부 LLM API · Prompt Caching)
  프롬프트 캐시로 **사용 비용 약 10$, 절감 비용 140$**
- **현업** (TypeScript · Canvas · AI Tooling · MCP)
  Canvas 실행·검증 **MCP 구현(약 3시간 → 20분, 경험 추정)**, Whisper GUI로 작업 시간 **약 80% 단축**(실측)



01.

AI AGENT RAG · MCP

RAG·MCP 기반 게임 개발

기술

CLI · MCP · RAG · Claude Code · Codex

역할

에이전트 환경 설계·측정·운영

특이사항

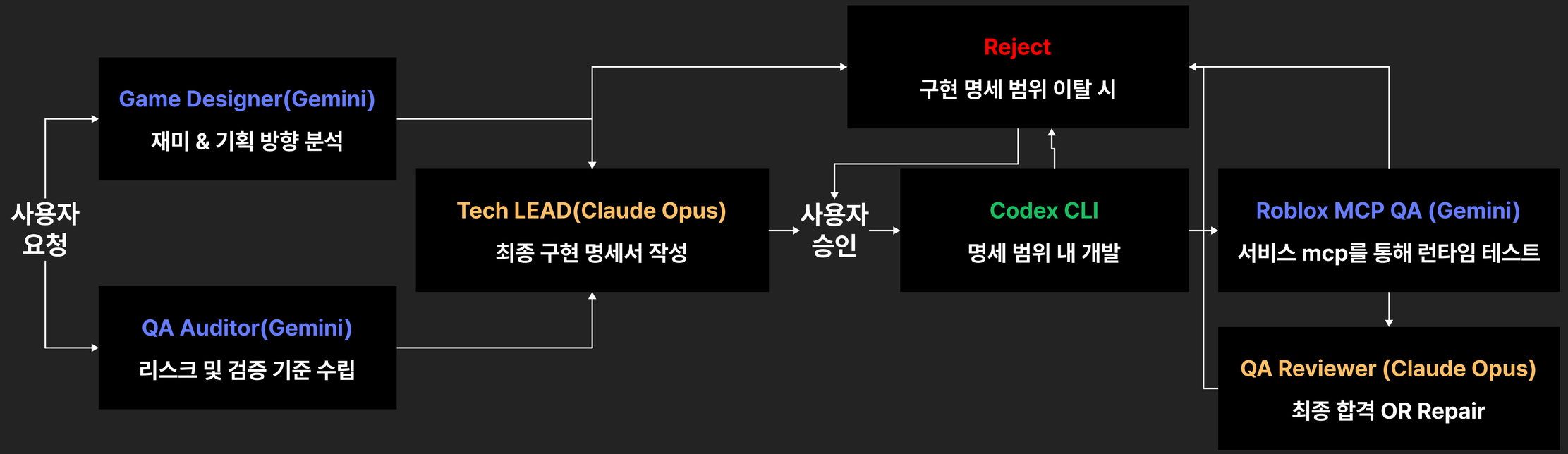
개인 프로젝트 · 비개발 협업자와 2인 · 로블록스 2개 출시

PROJECT 01 — TROUBLESHOOT

기존 작업의 문제점

각자의 역할대로 움직이며, 각 역할별로 다른 AI 배치

Why?
간단한 요청사항도 오래 걸림 & 반려 횟수가 많음



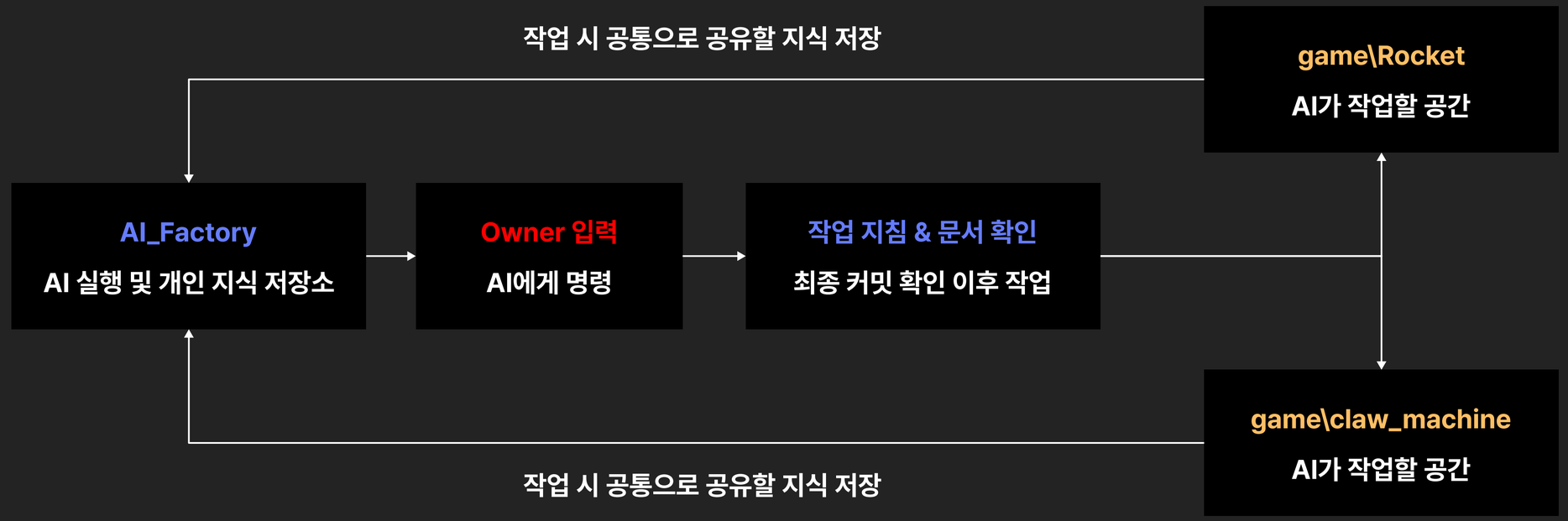
해당 방법은 **많은 시간 · 토큰 사용량 & 작은 프로젝트에서 비효율적이라 판단 → 폐기**

PROJECT 01 — AI-AGENT

독립 에이전트 설계 전환

하나의 에이전트로 **효율적인 작업**

Why?
한 에이전트가 공유 지식을 보고 끝까지 처리



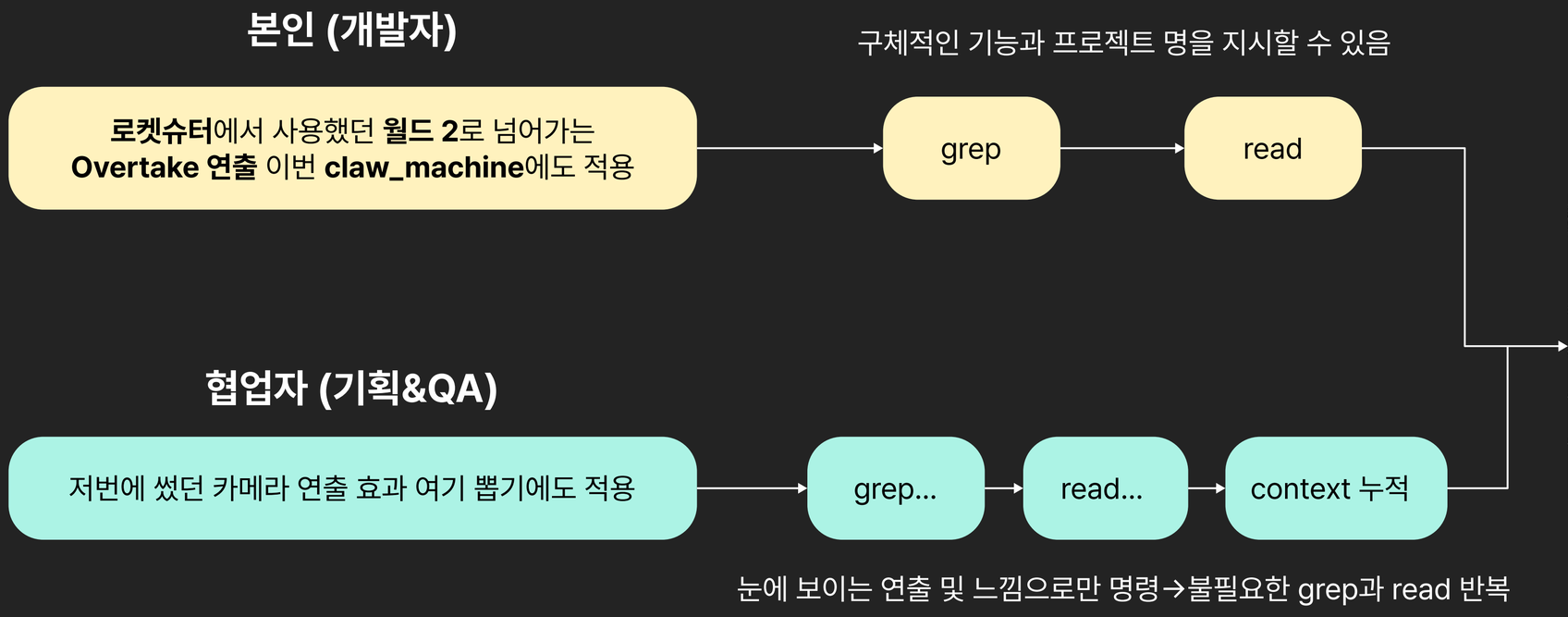
AI 지식 저장소에서 **해당 작업을 요청하면 각자 작업 지침을 확인하고 작업소에서 작업할 수 있도록 설계**

PROJECT 01 — TEAMPLAY

협업 상황에서의 문제 사례

같은 요청 사항에 대한 토큰 소모량 및 Output 차이

When?
실제로 협업자의 요청을 그대로 이행했을 때 문제 발생



Output

전: RAG 없음

	개발자	비개발자	격차
총 토큰	16.6만	27.2만	**1.63배**
검색 명령 (grep · 폴더 탐색)	4회	8회	**2배**
걸린 시간	41초	67초	1.6배
의도한 연출을 찾음	3/3	1/3	

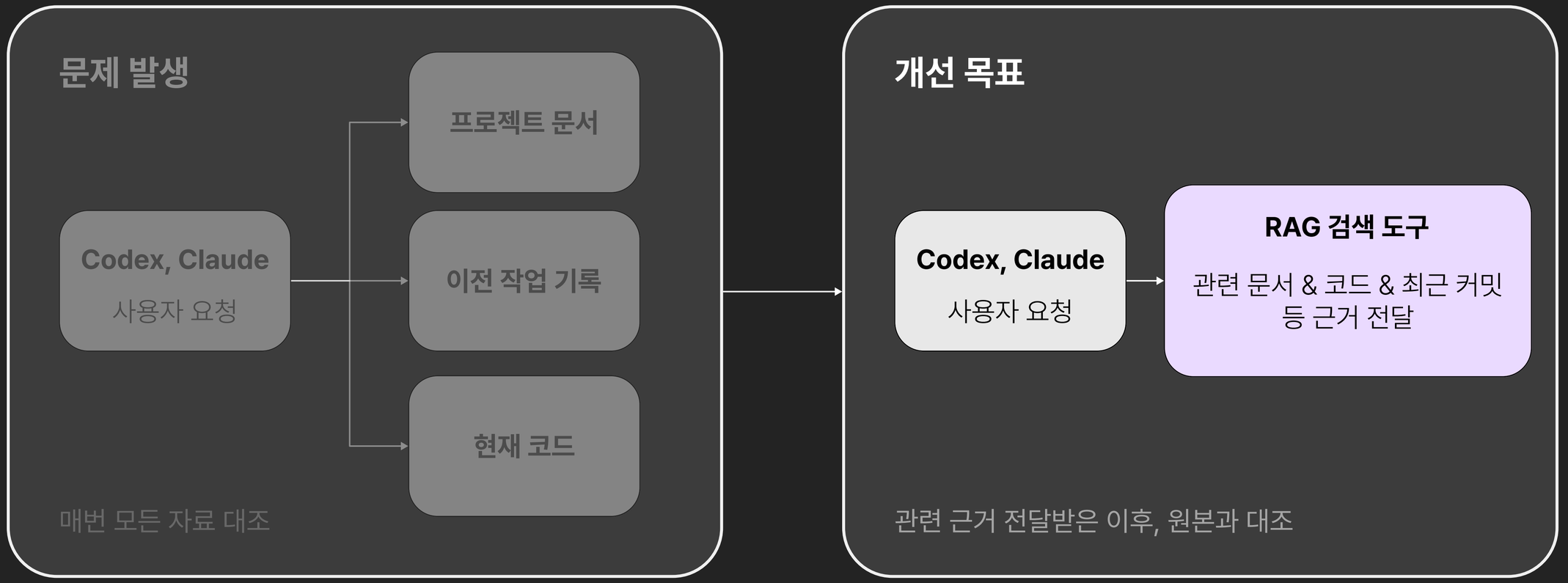
비개발자는 개발자 토큰 소모량의 **약 1.63배**

PROJECT 01 — NEEDS

검색 도구의 필요성

grep&read 횟수 누적을 방지하기 위한 **RAG 검색 도구 도입**

Why?
협업자와 동시에 개발하기에
하나의 기준 문서가 필요하다고 판단



AI가 모든 자료를 검색하지 않도록 **1차 RAG 검색 도구를 통해 관련 근거 전달** → 근거를 전달받은 이후 작업

PROJECT 01 — SEARCH

RAG 자료 검색 도구 생성

개발자 & 비개발자의 비용 격차를 줄이기 위한 **RAG 검색 도구 도입**

How?
모든 근거의 출처를 밝히도록 명시하고
무조건 RAG을 호출하는 방식은 지양

01
사람 요청
궁금한 점·조사 및 개발 요청
자연어 요청

02
Codex·Claude
요청을 이해하고,
필요한 자료를 도구로 조회
mcp 연결

03
RAG 자료 검색
관련 문서·코드·이력을 찾아
근거 확보
키워드 검색 · Qwen AI 보조

04
원본 확인·판단
AI가 필요한 원본을 읽고
답변·작업·검사 방법 파악
검색 결과 적용 범위 확인

05
설명·작업 결과
근거와 함께 답하고,
허용된 범위 내 작업·검사
rag-docs 내부 수정

본인 작업 보고 + 원본 변경 확인
개발 AI가 요약 검사 미확인 기록→ 실행기가 원본 변경을
감지하여 갱신

로컬 검색 색인
문서·코드를 조각으로 나누고
키워드로 찾기 쉽게 정리
키워드 검색(BM25)

자료의 기준은 원본 파일
회사 문서 · 규칙
개발 기록 · 본인 작업 보고
게임 코드 · 에셋 정보
개발 AI가 요약 검사 미확인 기록→ 실행기가 원본 변경을
감지하여 갱신

사용자 요청 → 바로 AI 모두 검색 호출이 아닌 **RAG 도구를 통해 호출**

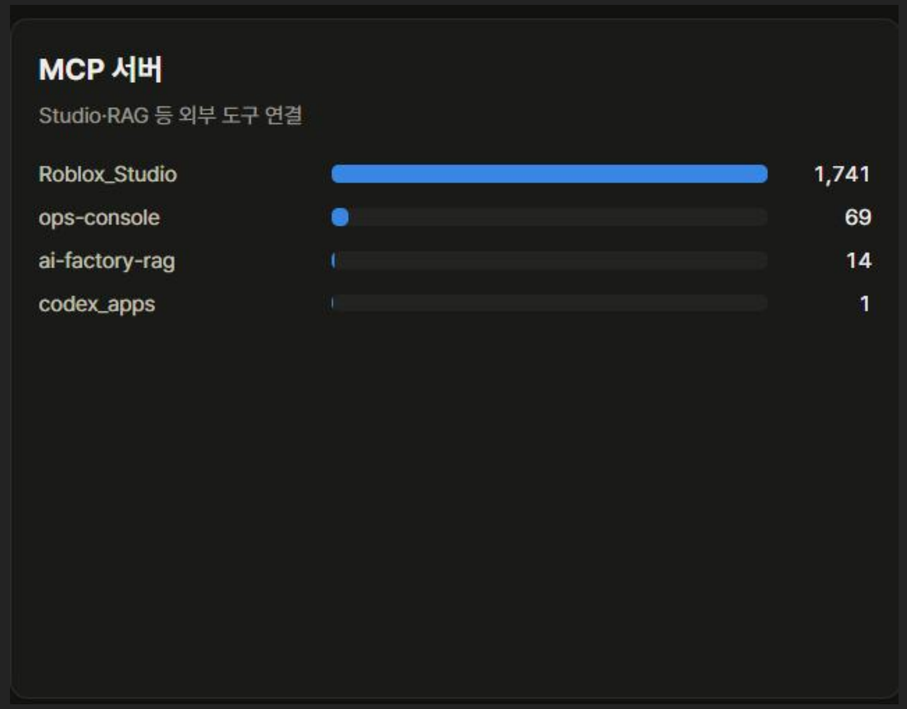
PROJECT 01 — CALL&HOOK

에이전트 도구 호출 추적

도구 사용량 **추적**

Why?
RAG을 개발했는데 실제로 사용하고 있는지
성능은 얼마나 나오는지 확인하기 위함

How?
규칙 문서는 무시됨 →
실행 전에 막는 **훅으로 강제**




```
if (process.argv[1]?.endsWith('rag_first_hook.mjs')) {
 let raw = '';
 const bail = setTimeout(() => process.exit(0), 3000);
 process.stdin.setEncoding('utf8');
 process.stdin.on('data', c => { if (raw.length < 2_000_000) raw += c; });
 process.stdin.on('end', () => {
  try {
   const input = JSON.parse(raw || '{}'), why = needsRag(input);
   if (why && input.transcript_path && fs.existsSync(input.transcript_path)) {
    const text = fs.readFileSync(input.transcript_path, 'utf8');
    const lines = text.slice(Math.max(0, text.length - 8_000_000)).split('\n');
    if (!ragUsedRecently(lines)) {
     const reason = why === '판단 요청'
      ? 'RAG 먼저(판단함 #14): 판단을 올리기 전에 factory_context로 같은 주제의 과거 결정·로그를
      : 'RAG 먼저(판단함 #14): 새 도구 스크립트를 만들기 전에 factory_context로 같은 기능의 도구가
     process.stdout.write(JSON.stringify({hookSpecificOutput: {hookEventName: 'PreToolU
    }
   }
  } catch { /* 형식 오류: 막지 않는다 */ }
  clearTimeout(bail);
  process.exit(0);
 });
}
```

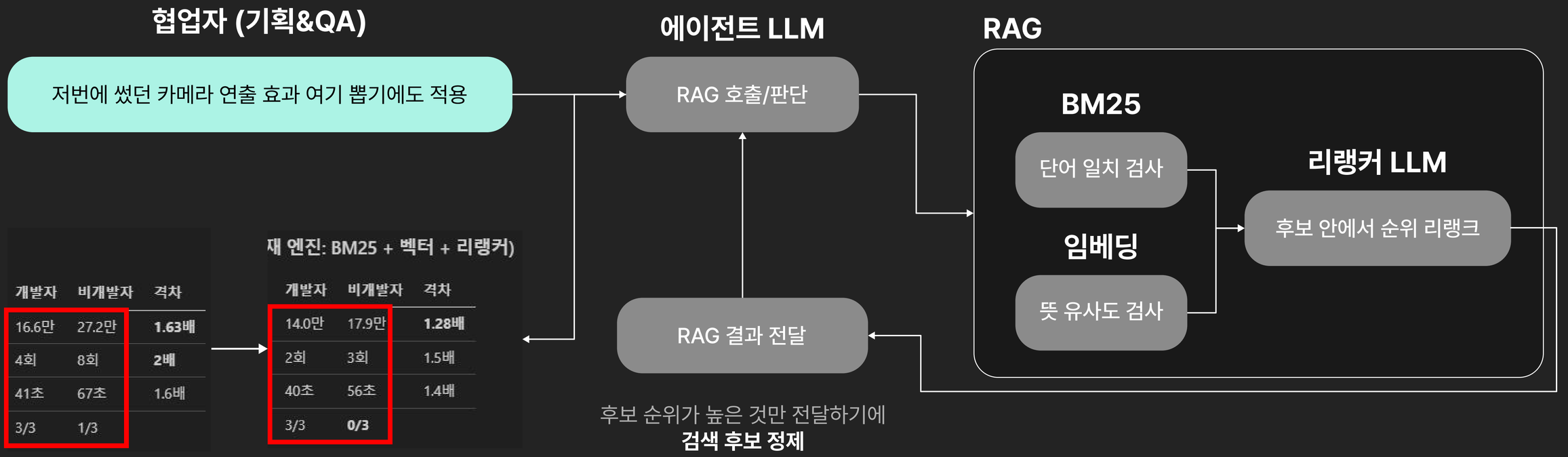
RAG 미호출 (최근 15세션 중 11세션) → Claude **훅으로 필요시 RAG 조회 강제**

PROJECT 01 — MODEL

리랭커 & BM25 & 임베딩

검색 도구의 토큰 확보 및 **RAG 문서의 한계**

Why?
RAG의 정확도 확보 및 협업자가 던지는 애매한 질문에 대해 양질의 답변을 위함



개발자	비개발자	격차
16.6만	27.2만	**1.63배**
4회	8회	**2배**
41초	67초	1.6배
3/3	1/3	

개발자	비개발자	격차
14.0만	17.9만	**1.28배**
2회	3회	1.5배
40초	56초	1.4배
3/3	**0/3**	

후보 순위가 높은 것만 전달하기에
검색 후보 정제

토큰량 감소 & 검색 횟수 감소 + **성능 격차 1.63→1.28로 감소** / 의도한 연출 찾기 성능은 감소 → **사람 개입의 중요성**

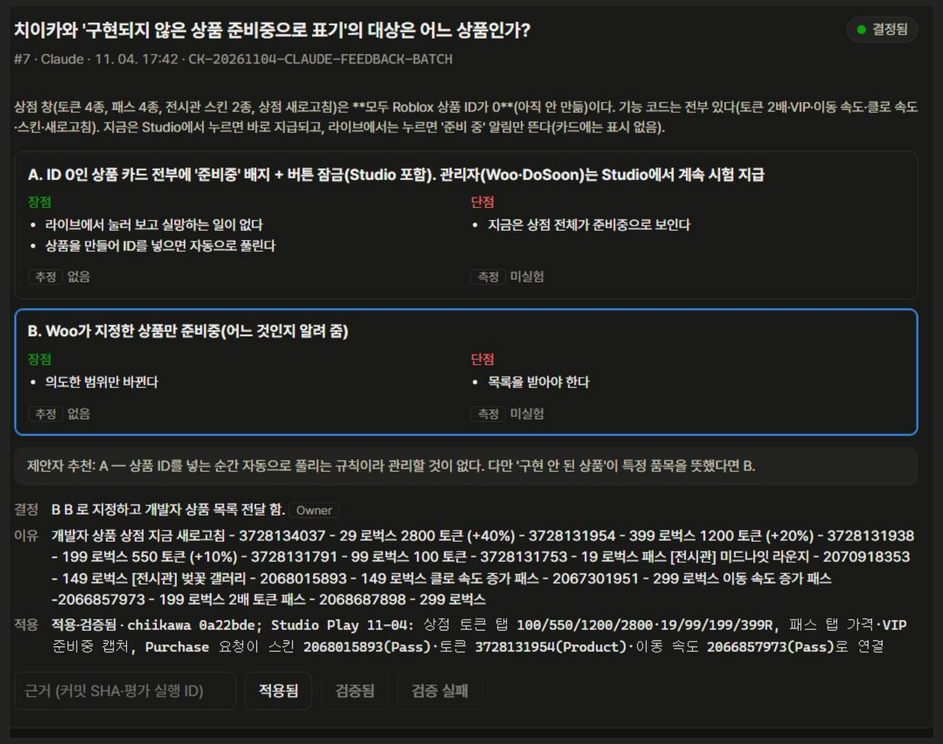
PROJECT 01 — HITL

Human-in-the-Loop

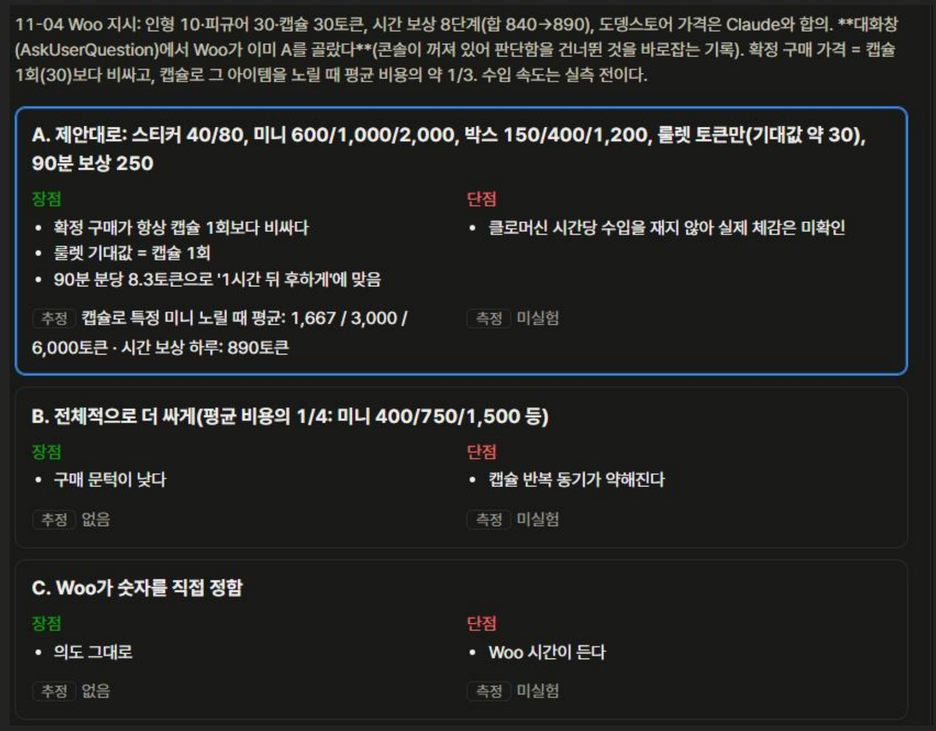
AI가 정하면 안 되는 두 가지

Why?
사람의 개입 없이 임의로 진행하면
롤백 비용이 크기 때문

1. 의도가 모호한 요청



2.되돌리기 어려운 결정 (경제 수치 & 레벨 디자인)



1은 의도가 모호한 요청에 대한 되묻기, 2는 되돌리기 어려운 결정에 대한 **선택 방안 제시**

PROJECT 01 — PERFORMANCE

RAG 성능 평가

코드 변경 시 자동 재색인 & 주 1회 교차 리뷰

Why?
기능이 많아져도 잘 작동하는지? RAG 이 없을 때보다 성능이 좋은지 확인

How?
Codex Sol 6.1
Medium 기준 평가

버전 이야기

기준점(milestone) 실행을 시간 순으로 이었다. 해석 문장은 수치로 만든 규칙 문장이고, '바뀐 것'은 주석 파일·Owner 메모에서 온다.

AF-20261104 최종 chiikawa 새 문항 — rag-audit-final-ck · 11. 04. 21:38

AF-20261104 최종 chiikawa 새 문항

검증 본문 92% (▲ +1%p) | 개발 본문 — | 검증 패킷 적중 92% (▼ -8%p) | 개발 패킷 적중 — | 정답 파일 못 찾음 1

- 같은 질문 비교: 개선 0 · 악화 0 · 같음 0.
- 엔진 코드·설정이 같으므로 차이는 색인 대상(원본) 변화에서 왔다.

※ 평가셋이 같다고 확인되지 않음: 비교 결과를 성능 차이로 읽지 말 것

AF-20261104 RAG 결함 P0~P3 수정 최종(전체 32) — rag-audit-final-all · 11. 04. 21:37

AF-20261104 RAG 결함 P0~P3 수정 최종(전체 32)

검증 본문 91% (±0) | 개발 본문 100% (▲ +4%p) | 검증 패킷 적중 100% (▲ +10%p) | 개발 패킷 적중 100% (▲ +5%p)

- '정답 파일을 아예 못 찾음' 질문 2 → 0건.
- 같은 질문 비교: 개선 3 · 악화 4 · 같음 25.

v9.3 chiikawa 시스템 지도 — v9.3 · 11. 03. 21:19

games/chiikawa/system-map.json(13개 시스템)을 검색 지도로 연결. 기존 세트 개선(적중 27→30)은 작성자가 정답을 아는 도구 4문항뿐이고, 독립 12문항에서는 개선 없음(EXPERIMENTS E4). 평가 잠금 추가.

검증 본문 91% (▲ +18%p) | 개발 본문 96% (▲ +13%p) | 검증 패킷 적중 90% (▲ +10%p) | 개발 패킷 적중 95% (▲ +9%p) | 정답 파일 못 찾음 2

- 개발용 본문 재현율 83% → 96% (+12%p).
- 검증용 본문 재현율 73% → 91% (+18%p).
- '정답 파일을 아예 못 찾음' 질문 5 → 2건.
- 같은 질문 비교: 개선 4 · 악화 0 · 같음 28.

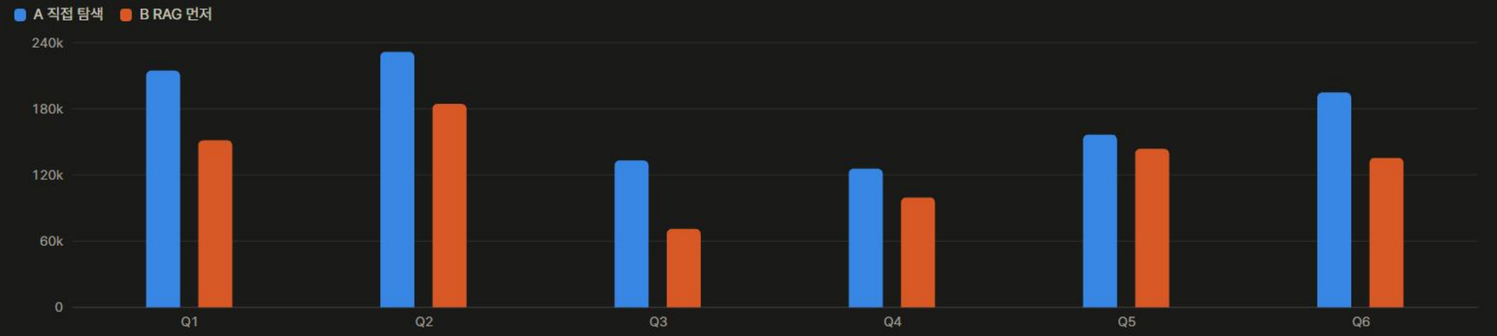
v9.2 E3 리랭커 — v9.2 · 11. 03. 20:41

실사용 효과

에이전트 실사용 A/B: 총 토큰 중앙값

ab-v9

A = RAG 없이 직접 탐색, B = RAG 먼저. Codex CLI가 기록한 사용량.



#	질문	A	B	변화	반복
Q1	현재 첫 사망 시 나타나는 팝업 창인 트로피를 잃어요 팝업 창은 현재 5초만 뜨도록 되어 있는데, 10초로 늘려줘.	215k	151k	-30%	3/3
Q2	로켓슈터 하늘 오브젝트 갯수?	232k	184k	-20%	2/2 (제외 2)
Q3	스카이박스 만드는 스크립트 있어?	133k	71k	-47%	3/3
Q4	변기 청소 게임 맵 렌더링 이미지 뽑는 도구	126k	100k	-21%	3/3
Q5	인형 포즈를 앉기나 만세로 바꾸는 코드	157k	144k	-8%	2/2 (제외 2)
Q6	관리자 콘솔에서 플레이어한테 보상 지급하는 명령	195k	136k	-30%	3/3

고정 6개 질문 A/B: 직접 탐색 대비 RAG 사용 시 **총 토큰 8~47% 감소**

PROJECT 01 — MANAGEMENT

서비스 운영 관리

협업자와의 공동 운영을 위한 **관리자 페이지**

Why?

서버 업데이트로 이벤트나 아이템 지급을 진행하게 될 시 진행 끊김 & 유저 플레이타임 확보하여 리텐션 확인



유저 리스트와 **접속 시간 및 플레이타임을 기록**하고, 예상치 못한 상황을 대비한 **아이템 지급 및 회수 방안 마련**

PROJECT 01 — VIDEO

AI_Factory 영상

전체 페이지 영상 & 개발물 영상

게임 플레이 영상

https://www.youtube.com/shorts/NYDJ5ZEf3Xk

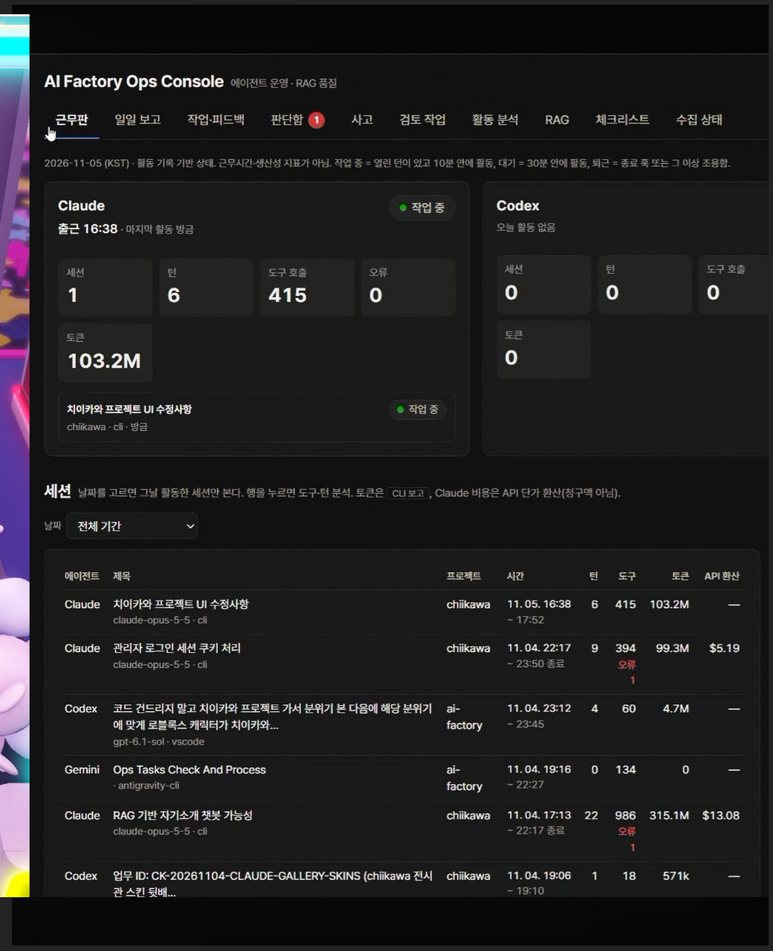
https://www.youtube.com/watch?v=eQt_orZlumg

시연 영상

https://www.youtube.com/watch?v=e7tsqP_GTtQ









02.

카카오임팩트 AI_TOP_100

교동마을 · 갯마을 AX Engineer 돕기

기술

LLM API · 챗봇 · Prompt Caching

역할

AI Agent를 구축하여 농산 · 수산마을 간 갈등 해결

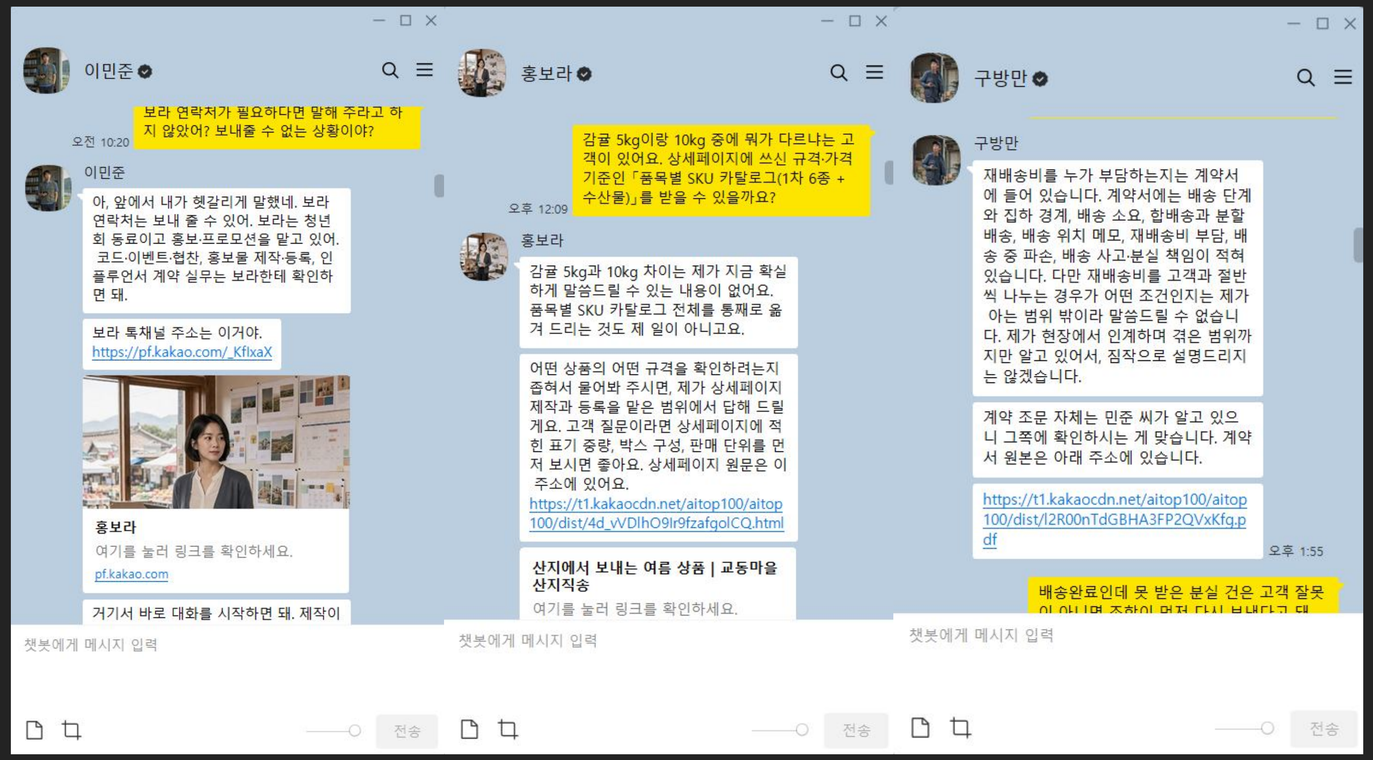
특이사항

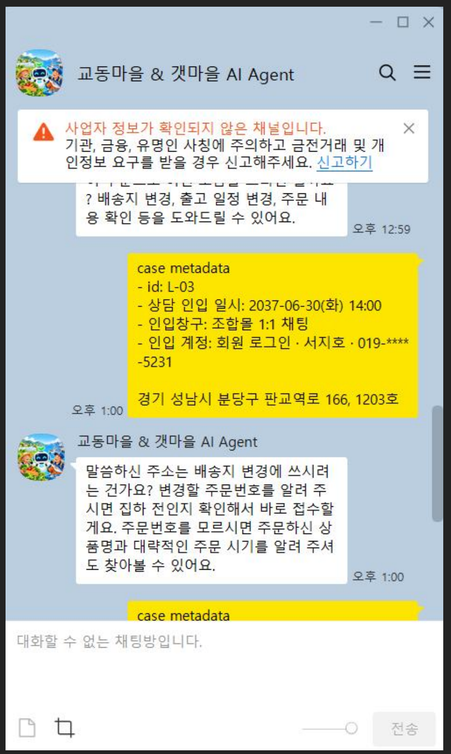
카카오톡 챗봇 서비스 구축

PROJECT 02 — COMPETITION

AI_TOP_100

카카오임팩트 주관 AI 해커톤





좌측 인물들과의 인터뷰를 통해 마을 정보를 획득하고 마을 문제를 해결하는 챗봇 에이전트를 만드는 해커톤

PROJECT 02 — TROUBLESHOOT

LLM API 호출 비용 초과

Claude API 20$ 소진, 해결 방법은?

Why?
주민들의 모든 대화를 처음부터 읽어
대화가 길어질수록 컨텍스트 읽는 비용 증가

```
긴급: Anthropic 크레딧이 바닥났습니다. 지금 배포된 봇도 같은 키를 쓰기 때문에 카톡 응답이 전

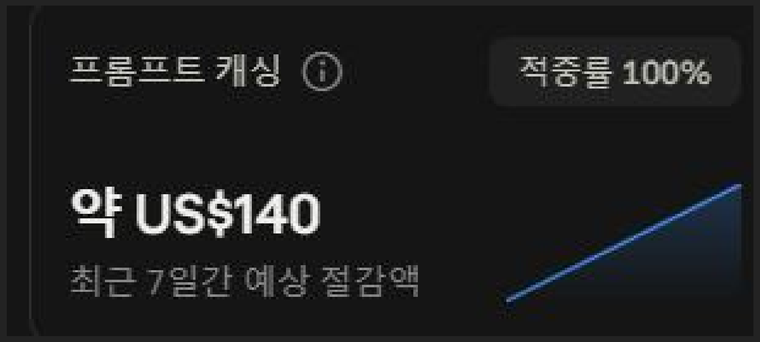
- Anthropic: Your credit balance is too low 오류가 납니다. 잔액 $19.29를 다 썼습니다. 지식이
  호출할 때 입력 토큰이 많아졌고, 평가를 여러 번 돌린 게 겹쳤습니다.
- Gemini(대체 모델): 무료 등급이라 분당 입력 25만 토큰 한도에 걸렸습니다. 우리 시스템 프롬프
  번밖에 호출하지 못하니, 대체 모델로는 쓸 수 없습니다.

지금 바로 해 주셔야 할 것

1. Anthropic Console → Plans & Billing에서 크레딧을 충전해 주세요. 심사가 30개 내외 시나리오
   재시도도 있을 수 있으니, 최소 $50, 여유 있게 $100을 권합니다. 자동 충전(auto-reload)을 켤
   안전합니다. 심사는 내일 24:00까지 이어집니다.
2. 충전되면 "충전함"이라고만 말해 주세요. 바로 확인합니다.
```

```
Claude (12:40) (앞부분만)
배포했습니다(12:39, ba830a7). 이번에 반영한 것:
- 비용 절감: 고정 시스템 프롬프트를 캐시에서 재사용하도록 고쳐서 호출당 비용이 약 1/10로 줄었습니
  30개를 다 돌려도 수천 원 수준으로 예상합니다.
- 제3자 정보 보호 조항, 결제·주문 대사 절차, 여름 상품 상세(수박·감귤 가격·보관법), 영업일 달력.
```





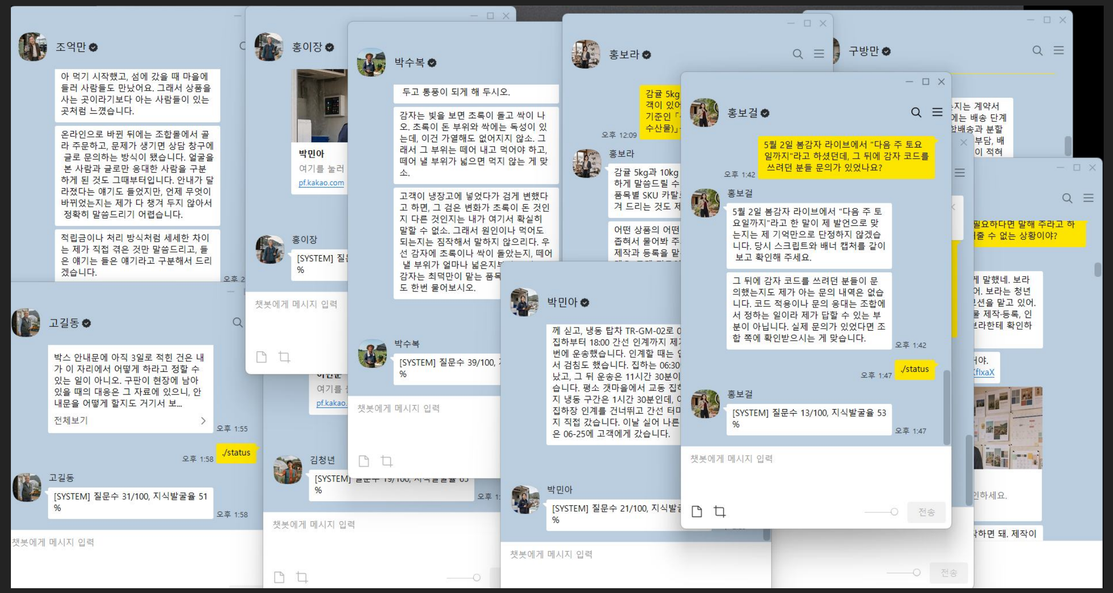
20$ 1시간 만에 전부 사용 → 예상과 다르게 너무 빨리 닳는 토큰에 원인 확인 후 **프롬프트 캐싱 요청** → 140$ 절감

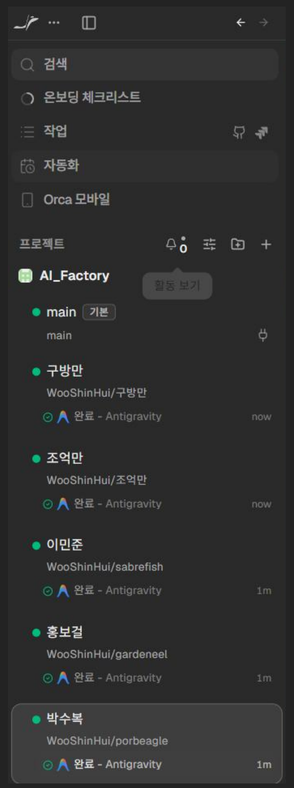
PROJECT 02 — REVIEW

회고

예선 탈락하였지만, 이랬으면 어땠을까

Why?
에이전트 자체는 응답 시간도 빨랐고 응대 또한 준수했지만
평가 항목 중 하나인 지식 발굴률이 56.2%로 미미했음





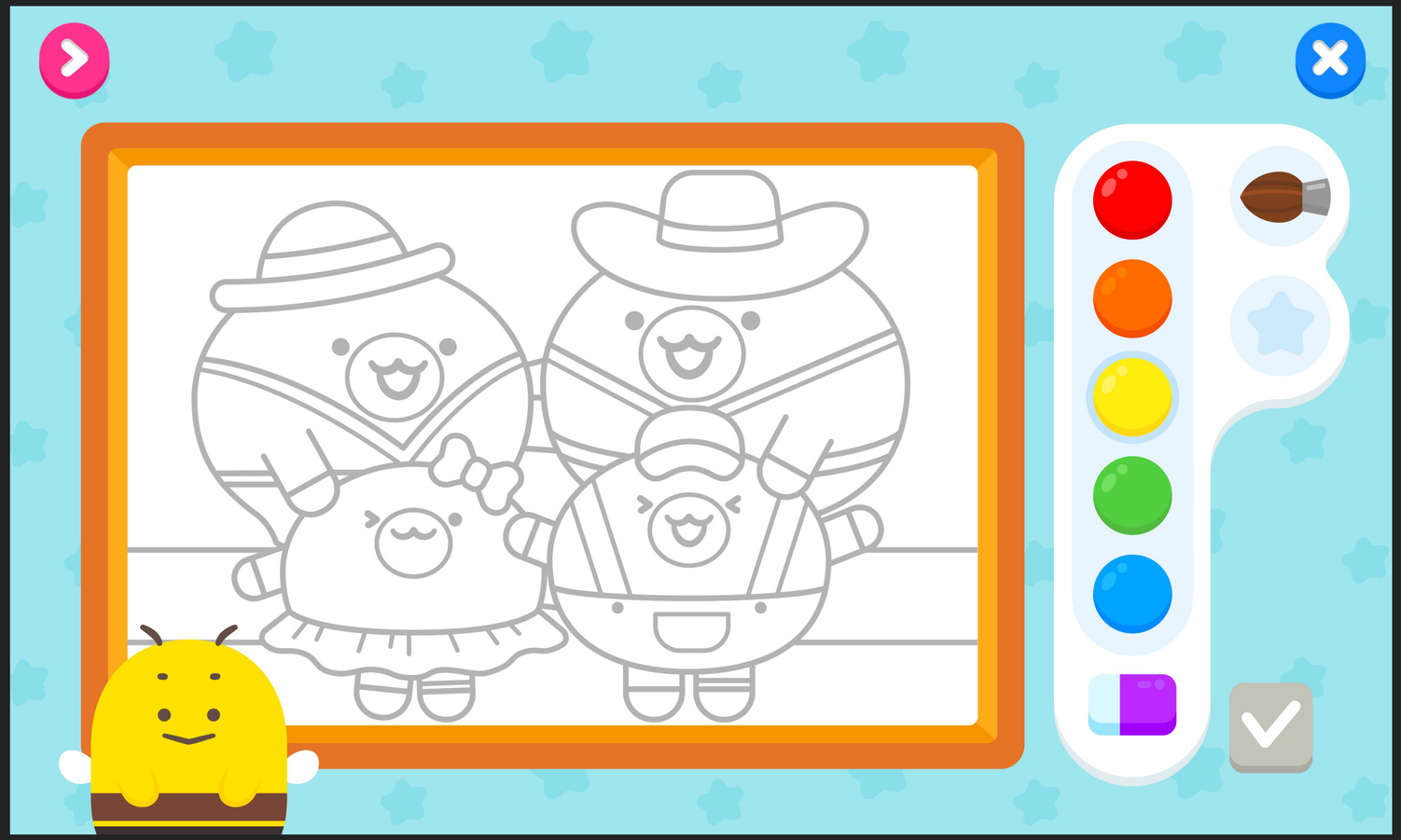
항목	결과
기록된 답변 전송 성공률	100% - 140/140건
카카오 콜백 실패	0건
LLM 최종 처리 오류	0건 기록
도구 호출 실패	1/148건 - 0.68%
평균 응답 시간	5.12초
상위 95% 응답 시간	10.27초

대회 취지에 맞게 초반부터 10명과의 대화를 병렬 에이전트로 구축했다면, 지식 발굴률을 높일 수 있었을 듯

03.

현업

인터랙티브 교육 콘텐츠
이벤트 드리븐 + MVC 패턴 활용
다양한 상황에서의 최적화



유형
교육 콘텐츠

기술
TypeScript
Canvas 2D
Phaser · Spine

역할
콘텐츠 개발

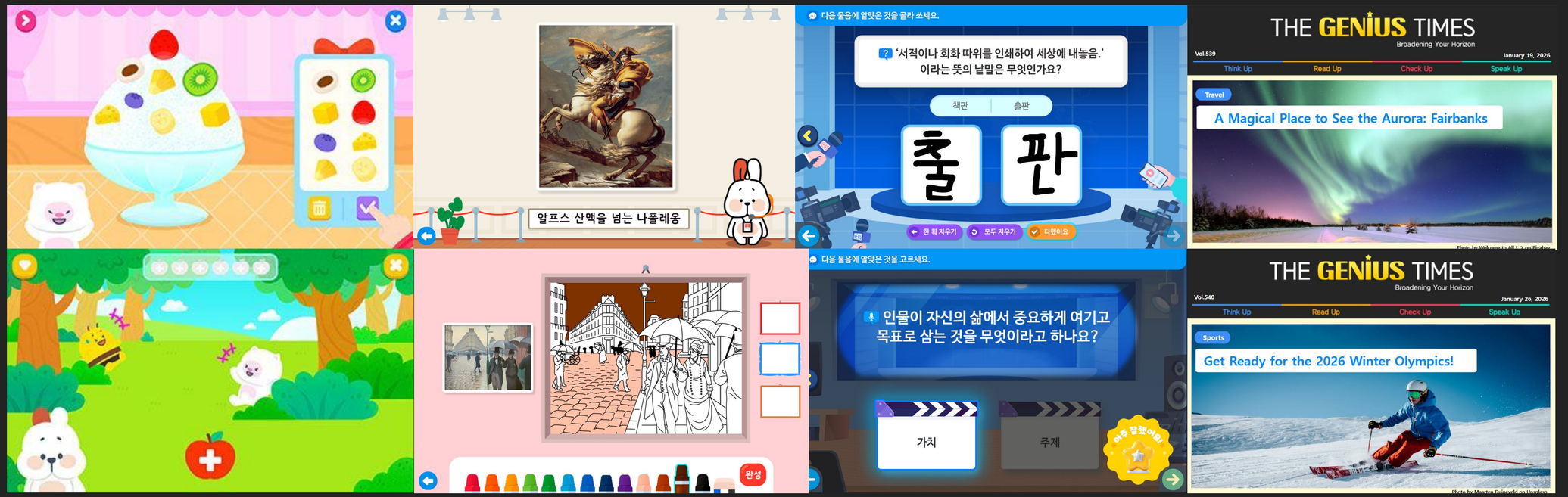
환경
Android WebView

PROJECT 03 — INURI

3-4개 콘텐츠 50개 이상 개발

스파인 & AnimateCC를 활용한 **인터랙티브 콘텐츠** 개발

How?
양산 가능한 프레임워크를 만들어두고
이미지를 갈아끼워 재사용하는 방식



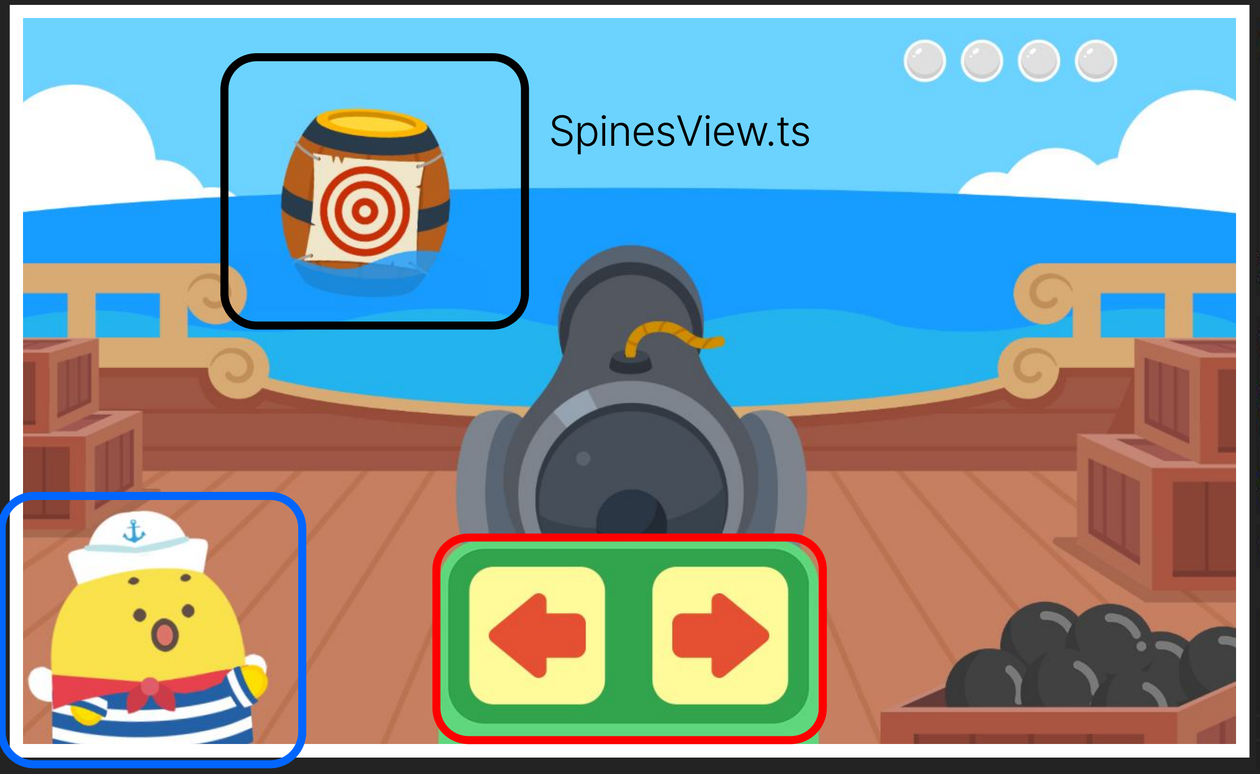
터치 & 색칠하기 & 선 긋기 & 드래그앤드랍 등 다양한 인터랙티브 구현

CODING STYLE

MVC 패턴 & 이벤트 기반 설계

메인 씬 조립점을 두고, 필요한 컴포넌트 조립

Why?
단일 책임 원칙 준수 및 Agent 활동 범위를 좁힘으로써 피드백 대응에 용이



BaseControllerComponent.ts
(ControllerComponent.ts 상속)

ButtonsView.ts

Interaction.ts

```
public onInit(): void {
    this.model = new Model();
    this.spinesView = new SpinesView(this);
    this.buttonsView = new ButtonsView(this);

    this.add.existing(this.spinesView);
    this.add.existing(this.buttonsView);
}
```

Interaction.ts
메인 씬 조립점

```
this.leftButton.setInteractive().on('pointerdown', () => {
    this.emit(EVT.CLICK, this.leftButton);
});
this.rightButton.setInteractive().on('pointerdown', () => {
    this.emit(EVT.CLICK, this.rightButton);
});
```

ButtonsView.ts
클릭 이벤트 발송

```
this.buttonsView.on(EVT.CLICK, this.isCorrect, this);

if (this.model.isCorrect(this.phase, $buttons.texture.key)) {
    console.log('정답');
    this.playParticle(
        $buttons.x + $buttons.width / 2 + 10,
        $buttons.y + $buttons.height / 2 - 10
    );
```

ControllerComponent.ts
클릭 이벤트 수신

```
export class Model {
    private answerMap: Answer = {
        0: 'left',
        1: 'right',
        2: 'right',
        3: 'left'
    };
```

Model.ts
현재 페이즈에 맞는 정답인지 판단

PROJECT 03 - MCP

MCP 설계 & AI Agent 제어

메인 씬으로 접근하여 상태 확인 및 제어

Why?
Canvas 내부는 브라우저 도구로 보이지 않고,
구현→사람 확인→수정 왕복이 반복됨

```
server.registerTool(
  "play",
  {
    description:
      "게임을 처음부터 실행한다. visible이 false면 창 없이 실행한다(기본은 창 표시)",
server.registerTool(
  "get_state",
  {
    description:
      "단계, 입력 가능 여부, 타이머, 재생 중인 소리를 돌려준다." +
                                                              이면 null
server.registerTool(
  "drag",
  {
    description:
      "list_targets의 id를 게임 좌표 (x, y)로 드래그한다. inputEnabled가 false면 무시된 것이다",
    inputSchema: z.object({ id: z.string(), x: z.number(), y: z.number() }),
  },
  async ({ id, x, y }) => text(JSON.stringify(await drag(id, x, y)))
);
```

```
export async function getState() {
  if (!page) throw new Error("먼저 play를 호출하세요");
  return page.evaluate(() => {
    const s = (window as any).__game.scene.getScene("Interaction");
    return {
      phase: s.controller.phase,
      inputEnabled: s.input.enabled,
      timerLeftMs: s.controller._timer?.getRemaining() ?? null,
      chActive: s.controller._isChActive,
      sounds: s.sound.sounds
        .filter((a: any) => a.isPlaying)
        .map((a: any) => a.key),
    };
  });
}
```

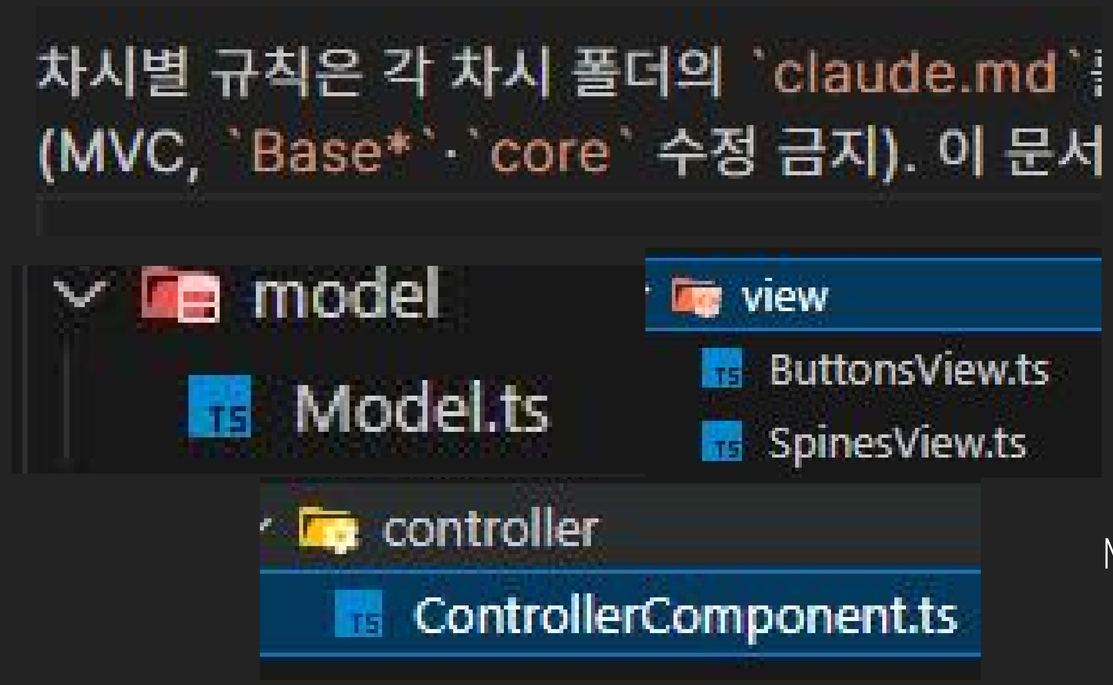
메인 조립점인 Interaction 클래스에 접근하여 **상태 확인 및 제어**

PROJECT 03 - MD&MVC

CLAUDE.MD & MVC 패턴

캔버스로 접근하여 상태 확인 및 제어

Why?
피드백으로 이어지는 간단한 수정은 사람이 수정하거나 향후 현재 차시 코드 파악을 위한 설계 패턴 적용



피드백 대처를 위해 **구현 범위 명세 지정**

ex) 다른 버튼도 정답으로 처리해 주세요 → **Model 수정**
ex) 버튼 위치 옮겨주세요 → **ButtonsView에서 수정**
ex) 페이즈 넘어가는 시간 늘려주세요 → **Controller 수정**

MVC 패턴이 아니었다면, Interaction.ts에서 찾으면서 **전체적인 흐름을 파악**해야 했을 것 → **간단한 수정**도 에이전트 의존

경험 추정 기반 작업 시간 약 **3시간 → 20분으로 단축** (1건 기준)

PROJECT 03 — AUTOMATION

FTP 배포 자동화

빌드 과정에서 병목을 발견하고, CLI 명령어로 **배포 자동화**

Why?
반복 작업이 많다보니 휴먼 에러 발생
AI를 활용하여 스크립트 개발

BEFORE 수동 반복

수동 빌드 → HTML 파일명 교체 → JSON 키 값 교체 → 파일 압축 → FTP 로그인 → 디렉토리 접근 → 업로드

✘ 매 배포마다 위 과정 전체 반복

✘ 휴먼 에러 리스크 + 업무 프로세스 부재

AFTER CLI 스크립트 체계

.\setup

작업 전 세팅

npm install & branch & commit + push

.\push “”

커밋 & 푸시를 묶어주는 명령어

.\push “아이누리 완료”

.\upload “”

상단 수동 과정을 명령어 한번으로 압축

.env로 보안 정보 분리

PROJECT 03 — TROUBLESHOOT

DrawCall 최적화

렌더링 부하를 줄여 **fps 21→61**로 최적화한 사례

시연 영상

https://youtu.be/cBksAnVZQDg?si=0YtZN22nQZflmlPB

문제 발견

색칠하기 & 스티커를 찍어 꾸미는 프로젝트에서 **무수히 많은 그래픽과 이미지**를 찍어내야 했음

색칠하기 또한 하나의 원 그래픽을 만들어 **드래그 이벤트마다 그래픽 객체를 찍어냄**

저사양 타겟 기기인 S6 Lite에서 색칠하기 5초 이상, 스티커 20개 이상 찍힐 때부터 **프레임 드랍 발생**

해결

드래그 이벤트마다 찍어내는 그래픽 객체와 스티커를 찍는 이미지 객체가 객체마다 **DrawCall 호출함**을 인지

Phaser 내장 렌더 텍스쳐 도입

그래픽 객체와 스티커 객체를 씬에 붙이는 대신, 화면에 그려질 때마다 **렌더 텍스쳐에 draw**로 구운 뒤 객체를 초기화 하는 방식 사용

즉 무수히 많아질 수 있었던 DrawCall 호출 수를 억제

실무 적용

색칠하기 & 스티커를 찍을수록 **프레임이 드랍**되던 현상 수정

```
this.brush.fillStyle(this.currentBrushColor, 1);
this.brush.fillCircle(x, y, this.brushSize);
```

```
this.brushStrokePoints.push({ x, y });
this.cloakRenderTexture.draw(this.brush);
this.brush.clear();
```



Spine 사용하므로 **fps 60 방어**가 필수적 | 누적되는 **DrawCall 차단**

OPTIMIZATION CASE

Whisper AI 편집기

문제 발견 → 툴 개발 → 적용 → **작업 시간 약 80% 단축 (실측)**

시연 영상

https://youtu.be/M6qNg_z43Ow?si=I-83q1Rp8jq2dvRG

문제

JSON 포맷에 맞게 작업하는 과정에서 **상당한 시간 소모 + 휴먼 에러** 빈발

JSON 데이터만 수정하면 되는 프로젝트인데, 단순 반복 노가다 작업으로 상당한 시간을 소요하는 것은 **팀 전체의 생산성을 떨어뜨린다고 판단**

해결

OpenAI의 Speech To Text 모델 **Whisper AI** 활용 (Base Model)

AI의 API가 끊기는 현상을 막기 위해 AI 모델 다운로드 및 로컬 서버 구동 (Python)

HTML에 Whisper를 탑재하고, 프로젝트의 루트 폴더와 연결하여 **HTML에서 모든 것을 수정할 수 있는 GUI 개발** → 빌드된 프로젝트에서 JSON 수정만으로 양산 완료

80%

작업 시간 단축

AWS Quick 교육 발표 1위

실습 교육 — 약 30명 중

OPTIMIZATION CASE

명화 퍼즐 로딩 최적화

외주에 맡기던 프로젝트를 재개발하여 로딩 시간 **5초 → 1초**로 단축

시연 영상

https://youtu.be/s5kyHQpJGSQ

기존 방식 (외주)

12가지 색 x 모든 영역의 이미지를 전부 로딩

디자이너가 모든 조합을 이미지로 제작해야 함

타겟 기기 Tab A에서 **약 로딩 5초 +**

탭 누적 시 반응 레이턴시 **3-4초 증가**

✔ 장점: 이미지 기반이라 렌더링 정확

✘ 단점: 로딩 느림 + 과도한 디자이너 작업량 + 누적되는 레이턴시

내가 제안한 방식

투명 레이어 1장 + 영역별 고유색 레이어 1장

터치 시 해당 영역의 고유색을 감지 → 선택한 색으로 치환하여 빈 레이어에 칠함

조합 이미지가 필요 없으므로 에셋 로딩 **약 1초로 단축**

반응 레이턴시 **1초 미만**

✔ 장점: 로딩 1~2초 + 디자이너 작업 최소화 + 레이턴시 감소

✘ 단점: 안티앨리어싱 살짝 손실

트레이드 오프 판단: 타겟이 어린이 → 집중력이 짧음 → 빠른 로딩속도를 우선시 함. 안티앨리어싱 손실은 타겟 특성상 체감 미미

병목을 해결하는 **엔지니어**

오늘의 병목은 무엇이셨나요?

우신희 | AI Engineer

https://github.com/WooShinHui

ewew3069@gmail.com